

PROGRAMMATION ORIENTEE OBJET

1. BONNES PRATIQUES

La suite de ce cours reprend le site "Magasin" déjà vu au semestre 3, après avoir retiré les fonctionnalités qui ne seront pas utiles pour la suite :

- Contrôle d'accès par authentification (utilisation des variables de session).
- Mémorisation de la dernière page visitée (utilisation des cookies).
- Téléversement des images descriptives des articles (upload de fichiers).

Le site ainsi modifié a été enregistré dans l'archive `magasin.zip`, disponible sur le dossier Partage (la base de données reste inchangée).

Quelques autres modifications ont été apportées aux scripts afin de respecter les bonnes pratiques de la programmation de sites web en PHP :

- Une classe est déclarée dans un fichier dont le nom se termine avec l'extension `.class.php`.
Exemple : le fichier `modele/article.php` est renommé `modele/article.class.php`.
- Une classe est déclarée dans un fichier qui porte son nom (→ une seule classe par fichier).
Exemple : le fichier `modele/database.php` est renommé `modele/database.class.php`.
- Lorsqu'un fichier se termine avec la balise PHP fermante `?>`, cette dernière est supprimée.
Cette pratique empêche l'ajout de caractères parasites (espace, ...) à la fin du résultat renvoyé par le script PHP.

Récupérez Laragon sur le dossier Partage, puis installez le site dans un nouveau dossier publié par Laragon et vérifiez son bon fonctionnement :



Exercice :

Le routeur présent dans le fichier `index.php`, chargé d'exécuter l'action sélectionnée, se prête parfaitement à la mise en place d'une structure conditionnelle de type `switch` - `case`. En effet, cette structure est équivalente à une suite d'instruction `if` - `else if`.

Modifiez le fichier `index.php` de manière à mettre en place une structure `switch` - `case`.

Dans la suite de ce cours, nous allons appliquer la POO à toutes les parties du site (pour l'instant, seul le Modèle est écrit en POO).

2. VUES EN POO

Dans l'état actuel du site, les scripts chargés de générer les Vues présentent tous la même structure : ils initialisent les variables qui seront affichées dans le gabarit.

Cependant, seules les variables `$titre` et `$contenu` diffèrent entre les Vues...

Afin d'optimiser la programmation, nous allons centraliser la création des Vues dans la classe `Vue` (récupérez le fichier `vue.class.php`) :

```
<?php
/*****
Classe chargée de l'affichage des vues
*****/
class Vue {
...
}
```

La classe `vue` comporte :

- Un constructeur chargé d'initialiser la propriété `$fichierVue` à partir du nom de l'action demandée.
Exemple : Si `$action = "Clients"`, alors `$fichierVue` contient `"vue/vueClients.php"`.
- La propriété `$fichierVue` qui contient le nom du script permettant de générer le contenu et le titre de la page (stockés dans les variables `$contenu` et `$titre`).
- La méthode `afficher($data)`, chargée de générer la vue appropriée à partir des données transmises dans le tableau `$data`.

Au niveau du Contrôleur, l'appel des vues se fera en instanciant un objet de la classe `vue`.

Exemple : Affichage de la liste des clients dans le Contrôleur

```
$vue = new Vue("Clients"); // Instancie la vue appropriée
$vue->afficher(array("clients" => $clients)); // Affiche la liste des clients
dans la vue
```

Remarque : Le paramètre de la méthode `afficher()` est un tableau associatif.

→ Les valeurs stockées dans ce tableau associatif vont être extraites dans des variables nommées selon leur clé à l'aide de la fonction `extract()` (cf. manuel PHP).

→ Dans l'exemple ci-dessus, le résultat de `extract()` produira la variable `$clients`, qui sera utilisée dans le script `vueClients.php`.

→ Le script `vueClients.php` ne génère plus que les variables `$contenu` et `$titre`.

Exercice : Appliquez la POO pour afficher toutes les Vues.

Pour ce faire, il faudra :

- Recopier le fichier `vue.class.php` dans le dossier `vue/`.
- Adapter les fichiers `vuexxx.php` de manière à ce qu'ils initialisent uniquement les variables `$titre` et produise l'affichage qui sera stocké dans `$contenu` par la méthode `afficher()` de la classe `Vue`.
- Modifier le Contrôleur pour instancier un objet de la classe `Vue`, puis afficher la vue en passant les paramètres appropriés (cf. exemple ci-dessus). → Il faudra faire ces modifications pour chaque action.